SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

CO1 – ATI

PAVITHRA.K
192411027

2. stages of the Testing process

Requirement Analysis:

The tester studies the requirements of the ATM system such as card validation, PIN verification, balance checking, cash withdrawal and receipt generation

Test planning:

A test plan is prepared by defining the testing objectives, resources, schedule, tools and responsibilities for example, the plan includes testing valid and invalid PINs and insufficient account balances.

Test case Design

Test case are created based on the requirements

=> Entering a valid card and PIN
=> Entering an incorrect PIN
=> withdrawing an amount greater than the available balance

Test Environmental Setup

The ATM software, test database, card readers, cash dispenser simulator and network connection are prepared to perform testing under controlled conditions.

Test Execution:

The designed test case are executed and actual results are compared with expected results. Any mismatch is recorded as a defect.

Defect Reporting and Retesting

Detected defects are reported with details such as defect description, steps to reproduce, expected output, actual output, and severity.

After developers fix the defects, testers, execute the failed test cases again.

Regression Testing:

previously passed test cases are executed again to ensure that changes have not affected existing functionality.

Eg: ATM cash withdrawal

8. Cross-Browser Testing for an E-commerce platform

How Defects originate

Cross-browser defects may occur due to differences in browser rendering engines, CSS support, Javascript behavior, font rendering, and responsive Layouts. Developers may also use unsupported features or fail to test different

browser versions

Testing strategy:

i) identify Supported Browsers

Prepare a list of browsers and devices to be tested, such as Google chrome, Mozilla Firefox, Microsoft Edge, and Safari.

ii) Prepare test cases

create test cases for important functions, including:

=> User registration and login
=> product search and filtering
=> Adding products to the cart
=> Applying discount coupons
=> payment and checkout
=> order confirmation

Execute Functional Testing

perform the same test cases on all supported browsers and compare the actual results with the expected results.

perform UI and compatibility Testing

check fonts, images, buttons, menus, page alignment, colors, and responsive layouts on different screen sizes.

Report and Fix

Detects, records browser-specific defects, identify their causes, fix the issues, and retest the application.

Tools and Techniques:

Selenium: Automated browser-based test cases

Browserstack: supports testing on different browsers and devices

playwright: Automates testing across chromium, firefox, and webkit.

Compatibility testing: verifies consistent application behaviour.

10. Testing as an Engineering Activity

problems with random testing

Random testing without proper planning may result in:

⇒ Important requirements being used
⇒ Repeated execution of the same tests
⇒ Poor test coverage
⇒ Difficulty reproducing defects.

Redesigning the Testing process

Requirement Analysis:

study the software requirements and identify the functions that need to be tested.

Test planning:

Define the testing scope, objectives, resources schedule, responsibilities and risks

Structured Test case Design

prepare test case with inputs, preconditions, execution steps, expected results, and actual results

Apply Test Design Techniques:

Equivalence partitioning: Divides input data into valid and invalid groups.

Boundary value Analysis: Test values at and around the limits

Decision table testing: Test combinations of conditions and their expected outcomes.

State transition testing: checks how the system behaves when it moves between states.

Test Execution and Defect Reporting:

Execute test cases systematically and document failures with sufficient information for developers to reproduce them.

Retesting and Regression testing

Verify fixes and ensure that existing features continue to work correctly

Eg: Banking Application

15. Improving Test coverage and software reliability:

identifying test Gaps:

=> Missing test cases for certain requirements failure to test valid and invalid inputs

=> Lack of testing across different user roles

=> failure to test, exceptional conditions.

How proper coverage improve Reliability.

Requirement coverage

Every functional requirement should have corresponding test cases. A requirement traceability matrix can be used to map requirements to test cases.

Input Coverage:

Both valid and invalid inputs must be tested to identify incorrect behavior

Boundary value Testing:

Test values at the minimum and maximum limits and intermediate outside of those limits.

Decision and Branch coverage.

Test different decision outcomes and program branches to identify logical errors

Integration testing

.nority that different modules communicate and exchange data correctly

Regression testing:

Repeat existing tests after software modifications to ensure that previously working functionality remains constant.

Eg: online Banking Application

12. Process - Based Testing for consistency Across modules.

problems caused by inconsistent testing

=>Different modules allow different testing methods

=> some requirement may not be tested

=>Test reports may lack uniform information

=> Defects maybe repeated across modules

Applying a standardized Testing process

Testing standards -

Establish common rules for test planning, test case preparation, execution, defect reporting and test completion.

Analyze Requirements:

study the requirements of each module and identify its expected behavior

prepare standard Test cases

Use a common format containing test case, ID, objective input, preconditions, execution steps, expected result, actual result, and status.

Standardize Defect Reporting

Record each defect with a unique ID, description, severity, priority, reproduction steps, and status

Monitor Testing progress

Use test coverage, pass/fail results, defect, counts, and unresolved defects to measure to measure testing effectiveness

Eg: E-commerce Application